

การพัฒนา Network Application และการออกแบบ Application-Layer Protocol

1. วัตถุประสงค์ของโปรแกรม

โปรแกรมที่พัฒนาขึ้นมาสำหรับโปรเจกต์นี้คือ Collaborative Bucket List Application เป็น Network Application แบบ Client-Server ที่อนุญาตให้ผู้ใช้หลายคนเชื่อมต่อเข้ามายัง Server และใช้งาน Bucket List ร่วมกันผ่านทางเครือข่ายในระบบ

ผู้ใช้สามารถสร้างบัญชีผู้ใช้ด้วย Username และ Password จากนั้นเข้าสู่ระบบเพื่อใช้งาน Bucket List โดยสามารถเพิ่ม แก้ไข ดู และลบรายการต่าง ๆ ได้ นอกจากนี้ เมื่อมีผู้ใช้คนหนึ่งเพิ่ม แก้ไข หรือลบรายการ ผู้ใช้อื่นที่เชื่อมต่ออยู่กับ Server จะได้รับการแจ้งเตือนการเปลี่ยนแปลงดังกล่าว ทำให้ข้อมูลของ Bucket List สามารถใช้งานร่วมกันระหว่างผู้ใช้หลายๆ คนพร้อมกันได้

ตัวอย่างการใช้งาน เช่น ผู้ใช้สามารถเพิ่มรายการ

`/add "Visit Japan" "Travel around Tokyo and Kyoto"`

หรือเพิ่มรายการโดยไม่ระบุรายละเอียดก็ได้ โดยใช้คำสั่ง

`/add "Visit Japan"`

ในกรณีที่ไม่ได้ระบุ Description โปรแกรมจะเก็บเฉพาะชื่อรายการไว้ใน list

โปรแกรมยังมีระบบจัดการบัญชีผู้ใช้ โดยผู้ใช้ต้องลงทะเบียนก่อนจึงจะสามารถเข้าสู่ระบบได้ โดยการใช้คำสั่ง

`/register Alice password123`

จากนั้นจึงสามารถเข้าสู่ระบบด้วยการใช้คำสั่ง

`/auth Alice password123`

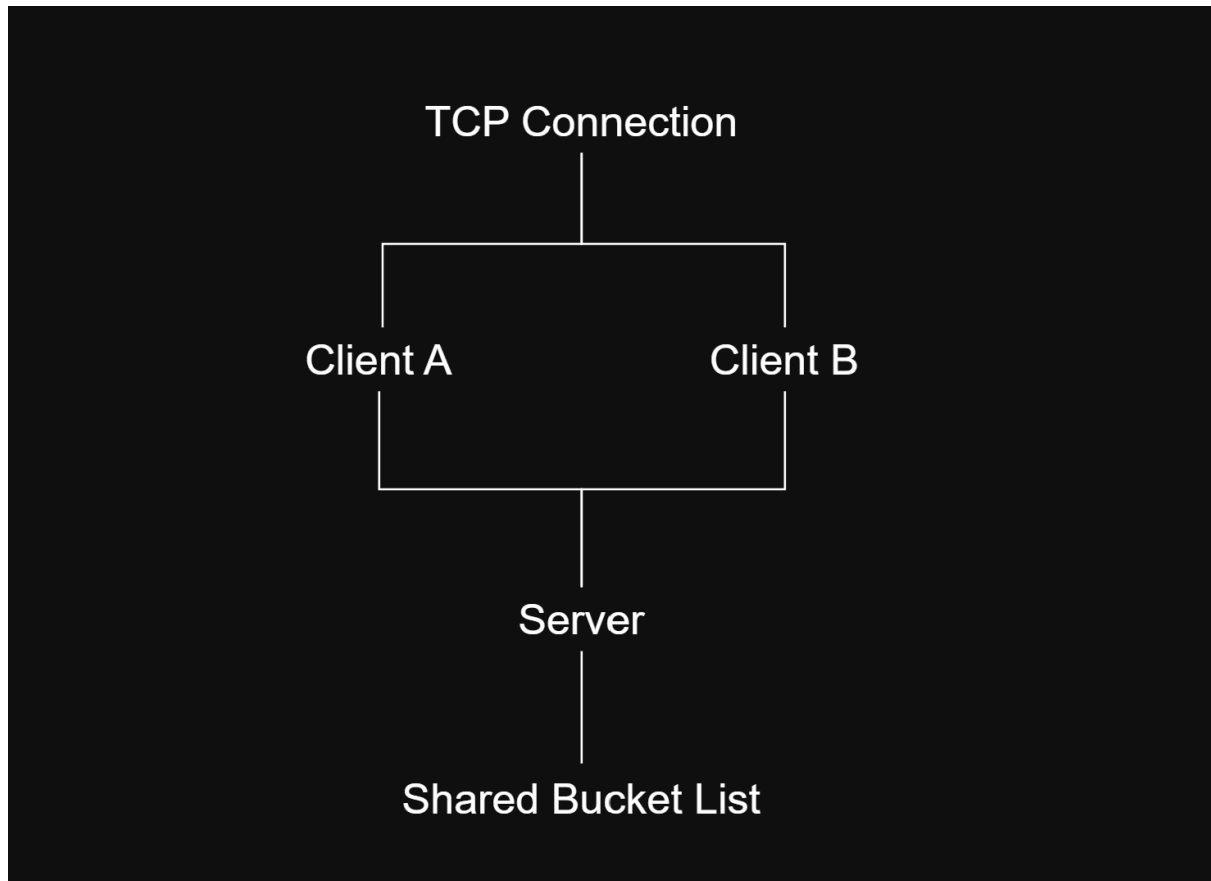
โดยคำสั่งเบื้องต้นต่างๆ จะถูกแจ้งให้ทราบคร่าวๆ โดยตัวโปรแกรมเมื่อผู้ใช้งาน (Client) เปิดใช้งานโปรแกรมทันที

2. Characteristics ของ Network Application

โปรแกรมมีคุณลักษณะดังต่อไปนี้

2.1 Client-Server Architecture

โปรแกรมใช้สถาปัตยกรรมแบบ Client-Server โดยมี Server เป็นศูนย์กลางในการจัดการข้อมูลและการสื่อสารระหว่าง Client โดยมีโครงสร้างคร่าวๆดังนี้



*Client แต่ละตัวจะส่ง Request ไปยัง Server เพื่อขอให้ Server ดำเนินการ เช่น สมัครสมาชิก เข้าสู่ระบบ เพิ่มรายการ หรือดูรายการ

*Server จะประมวลผล Request และส่ง Response กลับไปยัง Client

2.2 Multi-Client Support

Server สามารถรองรับ Client หลายตัวพร้อมกันได้ โดย Server จะสร้าง Thread สำหรับ Client แต่ละตัว ทำให้ผู้ใช้หลายคนสามารถใช้งานโปรแกรมพร้อมกันได้

2.3 User Registration and Authentication

ผู้ใช้ต้องสร้างบัญชีก่อนเข้าสู่ระบบ โดย Username จะต้องไม่ซ้ำกับบัญชีที่มีอยู่แล้ว Client การเข้าสู่ระบบต้องใช้ Username และ Password ที่ถูกต้อง

ระบบสามารถตอบกลับข้อผิดพลาดที่แตกต่างกัน เช่น

404 NOT_FOUND Account not found

หรือ

401 UNAUTHORIZED Wrong password

Client จึงสามารถทราบสาเหตุของการเข้าสู่ระบบที่ไม่สำเร็จได้อย่างชัดเจน

2.4 Shared Application State and Data

Bucket List ถูกจัดเก็บและจัดการโดย Server ดังนั้น Client หลายตัวจึงสามารถเข้าถึง Bucket List เดียวกันได้พร้อมกันในเวลาเดียวกัน

ตัวอย่าง:

User: Squid

```
[201 CREATED] ADD
{
  "item": {
    "id": 4,
    "title": "Acquire white Ink",
    "description": "Suspicious white liquid from Squid",
    "added_by": "Squid"
  }
}
>
```

User: Sato

```
[200 OK] USER_JOINED
{
  "username": "Squid"
}
>
[200 OK] ITEM_ADDED
{
  "item": {
    "id": 4,
    "title": "Acquire white Ink",
    "description": "Suspicious white liquid from Squid",
    "added_by": "Squid"
  }
}
>
```

2.5 Request-Response Communication

การทำงานของ Application ใช้รูปแบบ Client ส่ง Request ไปยัง Server และ Server ส่ง Response กลับมา โดยแต่ละ Request มีความหมายและรูปแบบข้อมูลที่กำหนดไว้ใน Application-Layer Protocol

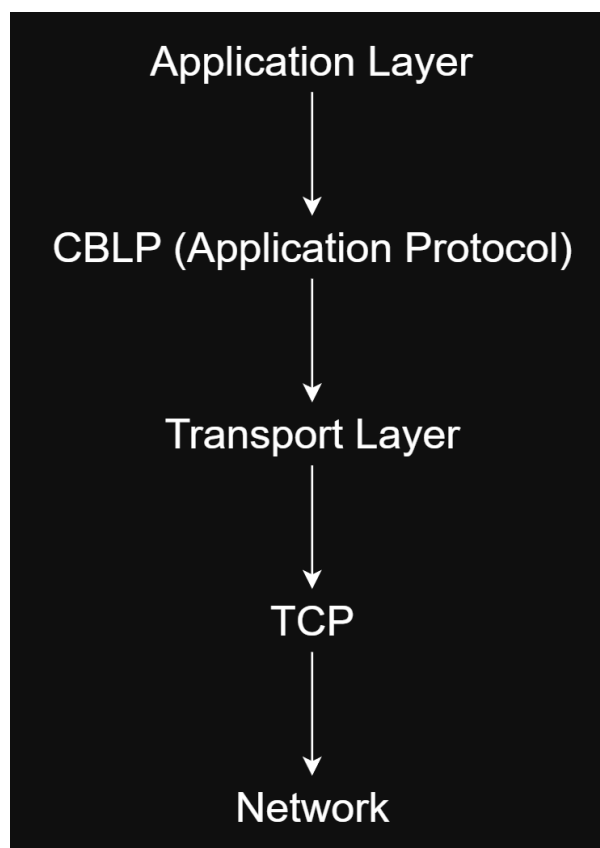
2.5 Real-Time Notification

เมื่อเกิดการเปลี่ยนแปลงกับ Bucket List เช่น การเพิ่มหรือลบรายการ Server สามารถส่ง Notification ไปยัง Client อื่นที่กำลังเชื่อมต่ออยู่ได้

*จากข้อมูลเบื้องต้น Application นี้จึงมีลักษณะเป็นรูปแบบ multi-client, stateful, collaborative Client-Server application ซึ่งผู้ใช้หลายคนสามารถทำงานกับข้อมูลชุดเดียวกันผ่าน Network ได้

3. Transport Layer Service Model

โปรแกรมเลือกใช้ TCP (Transmission Control Protocol) เป็น Transport Layer Service Model โดยมีโครงสร้างดังนี้



โดยเลือกใช้ TCP เนื่องจากข้อมูลที่ส่งระหว่าง Client และ Server มีความสำคัญต่อความถูกต้องของ Application เช่น Username, Password, คำสั่งเพิ่มรายการ และข้อมูลของ Bucket List และ มีคุณสมบัติที่เหมาะสมกับโปรแกรมที่สุด เพราะว่า

- **Reliable Data Transfer**
TCP มีการรับประกันว่าข้อมูลที่ส่งจะถูกส่งถึงปลายทางอย่างถูกต้อง หาก Packet สูญหาย TCP สามารถดำเนินการส่งข้อมูลใหม่ได้ โดยสำหรับโปรแกรมนี ความถูกต้องของข้อมูลสำคัญที่สุด
- **Ordered Data Delivery**
TCP รับประกันว่าข้อมูลที่ Application ได้รับจะอยู่ในลำดับเดียวกับที่ถูกส่ง ซึ่งเหมาะกับการสื่อสารแบบ Request-Response ที่โปรแกรมใช้ (Register Auth Add List Remove)
- **Connection-Oriented**
TCP มีการสร้าง Connection ระหว่าง Client และ Server ก่อนเริ่มส่งข้อมูล ทำให้เหมาะกับ Application ที่ Client จะติดต่อกับ Server อย่างต่อเนื่องตลอด Session
- **เหมาะกับข้อมูลที่ต้องการความถูกต้องมากกว่าความเร็ว**
Bucket List ไม่ใช่ Application ที่ต้องการ Latency ต่ำมาก เช่น Online Game หรือ Live Streaming ดังนั้นความถูกต้องและความน่าเชื่อถือของข้อมูลมีความสำคัญมากกว่าความเร็ว

4. Application Layer Protocol

Protocol ของ Application Layer มีชื่อว่า

CBLP – Collaborative Bucket List Protocol

CBLP เป็น Application-Layer Protocol ที่ใช้สำหรับการสื่อสารระหว่าง Client และ Server ของ Collaborative Bucket List Application โดย Protocol นี้ทำงานบน TCP

*ข้อมูลของ CBLP ใช้ JSON format ในการแลกเปลี่ยนข้อมูลระหว่าง Client และ Server

5. รูปแบบของ Request Message

Request จาก Client ไปยัง Server มีรูปแบบดังนี้

```
{  
  "request": "REQUEST_NAME",  
  "payload": {}  
}
```

โดย

request: ระบุประเภทของ Request

payload: ข้อมูลที่จำเป็นสำหรับ Request นั้นๆ

เช่น

```
{
  "request": "ADD",
  "payload": {
    "title": "Visit Japan",
    "description": "Travel around Tokyo and Kyoto"
  }
}
```

6. รูปแบบของ Request Message

Server จะตอบกลับ Client ด้วยรูปแบบ

```
{
  "status": 201,
  "phrase": "CREATED",
  "request": "ADD",
  "payload": {}
}
```

```
[201 CREATED] ADD
{
  "item": {
    "id": 1,
    "title": "Visit Japan",
    "description": "Travel around Tokyo and Kyoto",
    "added_by": "Alice"
  }
}
```

status: HTTP-like status code ที่แสดงผลลัพธ์

phrase: Status phrase ที่อธิบาย Status Code

request: Request ที่ Response ที่ตอบกลับ

payload: ข้อมูลผลลัพธ์ หรือ ข้อความ Error

*การใช้ Status Code ทำให้ Client สามารถแยกแยะได้ว่า Request สำเร็จหรือเกิดข้อผิดพลาดประเภทใด

7. CBLP Request and Response Messages

7.1 REGISTER

ใช้สำหรับสร้างบัญชีผู้ใช้ใหม่

Request

```
{
  "request": "REGISTER",
  "payload": {
    "username": "Alice",
    "password": "password123"
  }
}
```

สำเร็จ

```
{
  "status": 201,
  "phrase": "CREATED",
  "request": "REGISTER",
  "payload": {
    "message": "Account created",
    "username": "Alice"
  }
}
```

```
[201 CREATED] REGISTER
{
  "message": "Account created",
  "username": "Alice"
}
```

Username ซ้ำ

```
{  
  "status": 409,  
  "phrase": "CONFLICT",  
  "request": "REGISTER",  
  "payload": {  
    "message": "Username already exists"  
  }  
}
```

```
[409 CONFLICT] REGISTER  
{  
  "message": "Username already exists"  
}
```

8. AUTH

ใช้สำหรับเข้าสู่ระบบด้วยบัญชีที่ลงทะเบียนไว้แล้ว

Request

```
{
  "request": "AUTH",
  "payload": {
    "username": "Alice",
    "password": "password123"
  }
}
```

Login สำเร็จ

```
{
  "status": 200,
  "phrase": "OK",
  "request": "AUTH",
  "payload": {
    "message": "Authentication successful",
    "username": "Alice"
  }
}
```

```
> /auth Alice a123
>
[200 OK] AUTH
{
  "message": "Authentication successful",
  "username": "Alice"
}
```

Ijwu Account

```
{
  "status": 404,
  "phrase": "NOT_FOUND",
  "request": "AUTH",
  "payload": {
    "message": "Account not found"
  }
}
```

```
> /auth Amogus a123
>
[404 NOT_FOUND] AUTH
{
  "message": "Account not found"
}
```

Password ผิด

```
{
  "status": 401,
  "phrase": "UNAUTHORIZED",
  "request": "AUTH",
  "payload": {
    "message": "Wrong password"
  }
}
```

```
> /auth Sato a123
>
[401 UNAUTHORIZED] AUTH
{
  "message": "wrong password"
}
```

Account นั้นกำลังถูกใช้งานอยู่

```
{
  "status": 409,
  "phrase": "CONFLICT",
  "request": "AUTH",
  "payload": {
    "message": "Account is already logged in"
  }
}
```

```
> /auth Alice a123
>
[409 CONFLICT] AUTH
{
  "message": "Account is already logged in"
}
```

9. ADD

ใช้สำหรับเพิ่มรายการลงใน Bucket List

เพิ่ม List ที่มี Description

```
{
  "request": "ADD",
  "payload": {
    "title": "Visit Japan",
    "description": "Travel around Tokyo and Kyoto"
  }
}
```

ไม่มี Description

```
{
  "request": "ADD",
  "payload": {
    "title": "Visit Japan"
  }
}
```

*Server จะสร้าง Item ID และบันทึก Username ของผู้เพิ่ม List

Response:

```
{
  "status": 201,
  "phrase": "CREATED",
  "request": "ADD",
  "payload": {
    "item": {
      "id": 1,
      "title": "Visit Japan",
      "description": "Travel around Tokyo and Kyoto",
      "added_by": "Alice"
    }
  }
}
```

```
[201 CREATED] ADD
{
  "item": {
    "id": 1,
    "title": "Visit Japan",
    "description": "Travel around Tokyo and Kyoto",
    "added_by": "Alice"
  }
}
```

10. LIST

ใช้สำหรับขอรายการ Bucket List ทั้งหมด

Request

```
{
  "request": "LIST",
  "payload": {}
}
```

Response

```
{
  "status": 200,
  "phrase": "OK",
  "request": "LIST",
  "payload": {
    "items": [
      {
        "id": 1,
        "title": "Visit Japan",
        "description": "Travel around Tokyo and Kyoto",
        "added_by": "Alice"
      },
      {
        "id": 2,
        "title": "Learn Guitar",
        "description": null,
        "added_by": "Bob"
      }
    ]
  }
}
```

```
> /list
>
[200 OK] LIST
{
  "items": [
    {
      "id": 1,
      "title": "Visit Japan",
      "description": "Travel around Tokyo and Kyoto",
      "added_by": "Alice"
    },
    {
      "id": 2,
      "title": "Learn Guitar",
      "description": null,
      "added_by": "Bob"
    }
  ]
}
```

11. VIEW

ใช้สำหรับดูรายละเอียดของ Bucket List Item ตาม ID

```
{
  "request": "VIEW",
  "payload": {
    "id": 1
  }
}
```

Response:

```
{
  "status": 200,
  "phrase": "OK",
  "request": "VIEW",
  "payload": {
    "item": {
      "id": 1,
      "title": "Visit Japan",
      "description": "Travel around Tokyo and Kyoto",
      "added_by": "Alice"
    }
  }
}
```

```
> /view 1
>
[200 OK] VIEW
{
  "item": {
    "id": 1,
    "title": "Visit Japan",
    "description": "Travel around Tokyo and Kyoto",
    "added_by": "Alice"
  }
}
```

*กรณีไม่มี Item ดังกล่าว: **404 NOT_FOUND**

12. EDIT

ใช้สำหรับแก้ไขข้อมูลของ Bucket List Item

Request

```
{
  "request": "EDIT",
  "payload": {
    "id": 1,
    "title": "Visit Japan",
    "description": "Visit Tokyo, Kyoto and Osaka"
  }
}
```

Response

```
{
  "status": 200,
  "phrase": "OK",
  "request": "EDIT",
  "payload": {
    "message": "Item updated"
  }
}
```

```
> /edit 1 "Visit Japan" "Visit Tokyo, Kyoto and Osaka"
>
[200 OK] EDIT
{
  "message": "Item updated",
  "item": {
    "id": 1,
    "title": "Visit Japan",
    "description": "Visit Tokyo, Kyoto and Osaka",
    "added_by": "Alice"
  }
}
```

13. REMOVE

ใช้สำหรับลบ Item

Request

```
{
  "request": "REMOVE",
  "payload": {
    "id": 1
  }
}
```

Response

```
{
  "status": 200,
  "phrase": "OK",
  "request": "REMOVE",
  "payload": {
    "message": "Item removed"
  }
}
```

```
> /remove 1
>
[200 OK] REMOVE
{
  "message": "Item removed",
  "item": {
    "id": 1,
    "title": "Visit Japan",
    "description": "Visit Tokyo, Kyoto and Osaka",
    "added_by": "Alice"
  }
}
```

```
> /remove 3
>
[404 NOT_FOUND] REMOVE
{
  "message": "Bucket-list item not found"
}
```

กรณีไม่พบ ID: **404 NOT_FOUND**

14. CLEAR

ใช้สำหรับลบรายการทั้งหมดใน Bucket List

Request

```
{
  "request": "CLEAR",
  "payload": {}
}
```

Response

```
{
  "status": 200,
  "phrase": "OK",
  "request": "CLEAR",
  "payload": {
    "message": "Bucket list cleared"
  }
}
```

```
[200 OK] CLEAR
{
  "message": "Bucket list cleared",
  "removed": [
    {
      "id": 2,
      "title": "Learn Guitar",
      "description": null,
      "added_by": "Bob"
    }
  ]
}
```

15. USERS

ใช้สำหรับดูรายชื่อผู้ใช้ที่กำลังเชื่อมต่อกับ Server

Request

```
{
  "request": "USERS",
  "payload": {}
}
```

Response

```
{
  "status": 200,
  "phrase": "OK",
  "request": "USERS",
  "payload": {
    "users": [
      "Alice",
      "Bob"
    ]
  }
}
```

```
> /users
>
[200 OK] USERS
{
  "users": [
    "Alice",
    "Bob"
  ]
}
```

16. QUIT

ใช้สำหรับสิ้นสุด Session ของ Client

Request

```
{
  "request": "QUIT",
  "payload": {}
}
```

Response

```
{
  "status": 200,
  "phrase": "OK",
  "request": "QUIT",
  "payload": {
    "message": "Goodbye"
  }
}
```

```
[CONNECT] 127.0.0.1:58927
[DISCONNECT] Alice
```

17. Server Notification Messages

นอกจาก Request และ Response ปกติแล้ว CBLP รองรับ Server Notification เพื่อแจ้ง Client อื่นเมื่อ Shared Bucket List มีการเปลี่ยนแปลง ตัวอย่างเช่น เมื่อ Alice เพิ่ม Item:

```
{
  "status": 200,
  "phrase": "OK",
  "request": "ITEM_ADDED",
  "payload": {
    "item": {
      "id": 1,
      "title": "Visit Japan",
      "description": "Travel around Tokyo and Kyoto",
      "added_by": "Alice"
    }
  }
}
```

EX:

```
>
[200 OK] LIST_CLEARED
{
  "by": "Alice"
}
>
[200 OK] USER_LEFT
{
  "username": "Alice"
}
```

Notification ประเภทหลัก ได้แก่

USER_JOINED: มีผู้ใช้เข้าสู่ระบบ

USER_LEFT: มีผู้ใช้ออกจากระบบ

ITEM_ADDED: มีการเพิ่ม Item

ITEM_UPDATED: มีการแก้ไข Item

ITEM_REMOVED: มีการลบ Item

LIST_CLEARED: Bucket List ถูกล้าง

นายจักริน แสนภักดี

671040889